

JAVA LAB RECORD

Experiment No.	06
Date	23/09/26
Title	Implementation of Stack Methods in Java

Objective / Aim

To study and implement the commonly used methods of the Java Stack class, including push, peek, search, pop, and empty operations, and to understand the Last-In-First-Out (LIFO) principle.

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Keyboard and mouse
- Monitor/display

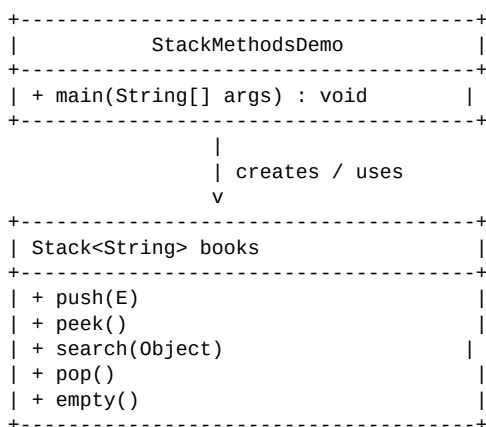
Software Requirements

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK): 17 or later
- IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Start the program.
2. Import the Stack class from java.util.
3. Create a Stack named books to store String values.
4. Insert elements into the stack using push().
5. View the top element using peek() without removing it.
6. Find the position of an element from the top using search().
7. Remove the top element using pop().
8. Check whether the stack is empty using empty().
9. Remove the remaining elements using pop().
10. Check the empty state again using empty().
11. Display all results.
12. Stop the program.

UML Class Diagram



Source Code

```
import java.util.Stack;
```

```

class StackMethodsDemo {

    public static void main(String[] args) {

        // Creating a Stack
        Stack<String> books = new Stack<>();

        // push(E item)
        books.push("Java");
        books.push("Python");
        books.push("C++");
        books.push("DBMS");

        System.out.println("After push(): " + books);

        // peek()
        System.out.println("Top element using peek(): " + books.peek());
        System.out.println("Stack after peek(): " + books);

        // search(Object o)
        System.out.println("Position of Python: " + books.search("Python"));
        System.out.println("Position of Java: " + books.search("Java"));

        // pop()
        String removed = books.pop();
        System.out.println("Removed element using pop(): " + removed);
        System.out.println("Stack after pop(): " + books);

        // empty()
        System.out.println("Is Stack empty? " + books.empty());

        // Pop remaining elements
        books.pop();
        books.pop();
        books.pop();

        System.out.println("Stack after removing all elements: " + books);

        // Checking empty again
        System.out.println("Is Stack empty now? " + books.empty());
    }
}

```

Completion and Execution Commands

1. Save the program as:
StackMethodsDemo.java
2. Open Command Prompt / Terminal and move to the folder containing the Java file.
3. Compile the program:
javac StackMethodsDemo.java
4. Execute the program:
java StackMethodsDemo
5. The results of all Stack operations are displayed on the console.

Output

```

After push(): [Java, Python, C++, DBMS]
Top element using peek(): DBMS
Stack after peek(): [Java, Python, C++, DBMS]
Position of Python: 3
Position of Java: 4
Removed element using pop(): DBMS
Stack after pop(): [Java, Python, C++]
Is Stack empty? false
Stack after removing all elements: []
Is Stack empty now? true

```

Result

Thus, the Java program was successfully implemented and executed to demonstrate the commonly used methods of the Stack class and the LIFO principle.